

COMPSCI5079 (M)

Cryptography & Secure Development

Dr. Nguyen Truong

Lecturer (Assistant Professor), SoCS, University of Glasgow

Email: Nguyen.Truong@Glasgow.ac.uk

Office: SAWB 221C, Sir Alwyn Williams Building, 18 Lilybank Gardens



Lecture 5:

Public-key Cryptography



Lecture 5:

Review





Outline

1. Overview

- Introduction
- A Big Picture

2. Diffie Hellman Key Exchange Protocol

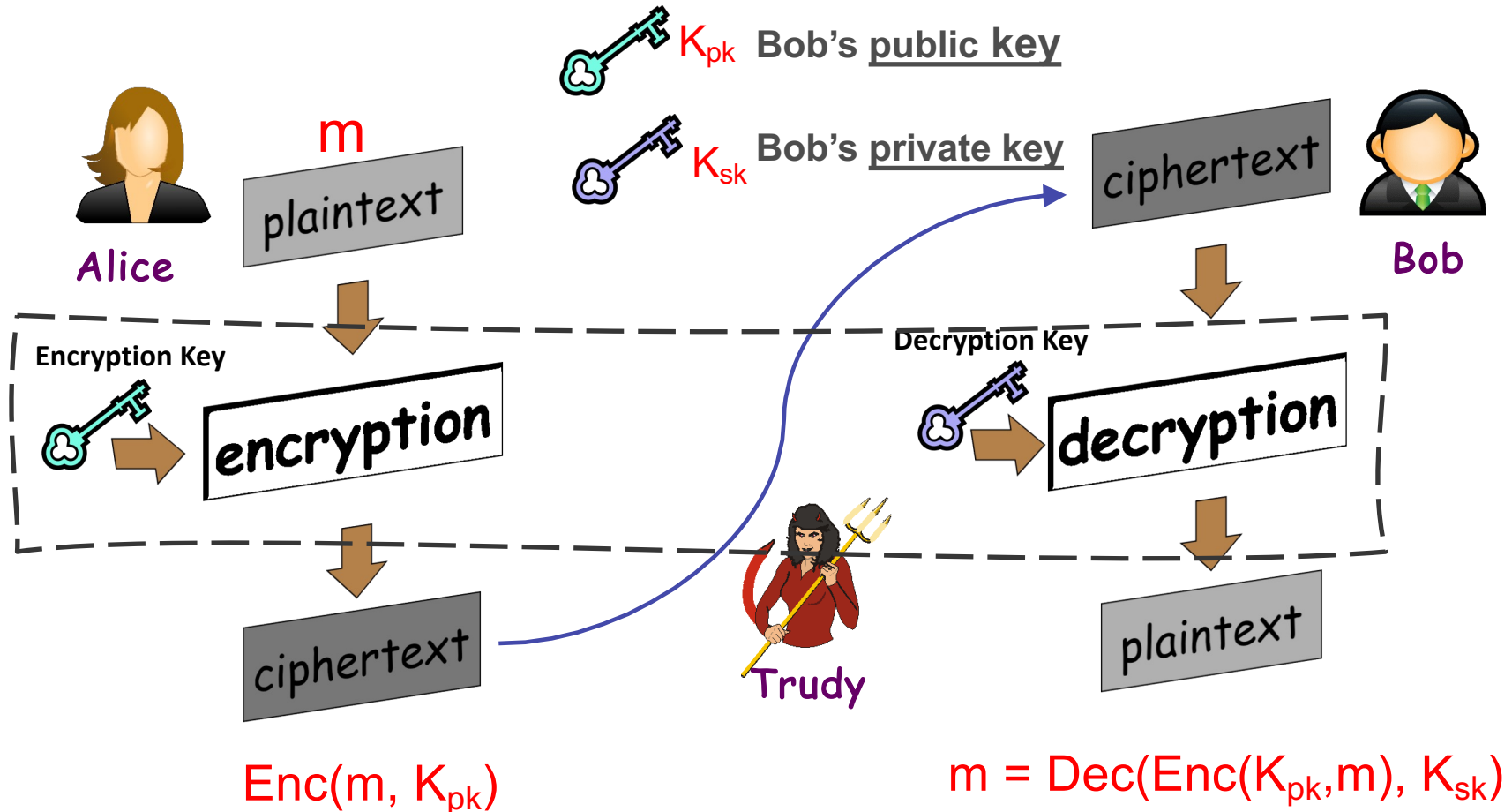
3. RSA Public key Cryptography

- KeyGen
- Encryption / Decryption
- Digital Signature

4. Elliptic Curves Cryptography



How Cryptography works





Public-key Cryptography: Requirements

- Each person owns a key-pair (K_{sk} , K_{pk}).
- K_{pk} revealed to the world; K_{sk} is kept secretly
- Given Public key K_{pk} , it is impractical to compute the private key K_{sk}
- Need to design functions **Enc** and **Dec** so that:

$$m = \text{Dec}(K_{sk}, \text{Enc}(K_{pk}, m))$$

- Note that: "Use public key first (encrypt), followed by private key (decrypt)" AND "Use private key first, followed by public key" should produce the same results.



Diffie-Hellman Summary

- Diffie Hellman Key Exchange protocol
 - Public channels can be used to create a confidential shared key function.
 - A trapdoor/One-way function should be used
 - Mix colour example
 - In practice: we use "Multiplicative Group" of "Integers modulo p ", where p is prime, and g is a primitive root modulo p



Diffie-Hellman Summary

- Multiplicative group of "Integers modulo p "

$$s = g^n \bmod p$$

- We have:

Easy: given g , n , & p , solve for s

Hard: given s , g , & p , solve for n

Why?

- And the property of:

$$g^{a*b} \bmod p = g^{b*a} \bmod p$$



RSA Summary

➤ RSA algorithm:

1. How to generate the key pair:

Public key: $pk = (e, n)$; Private key: $sk = (d, n)$

2. How to encrypt using the public key:

Compute ciphertext $C = M^e \bmod n$

3. How to decrypt using the private key:

Compute $M' = C^d \bmod n$
 $= (M^e \bmod n)^d \bmod n$
 $= M^{ed} \bmod n = M$



Digital Signature





Digital Signature Scheme

- Digital Signature: a data string which associates a message with some originating entity. One party generates a signature, and many parties can verify.
- Digital Signature Scheme:
 - A public/private key pair generation
 - A signing algorithm: takes a message and a (private) signing key, outputs a signature
 - A verification algorithm: takes a (public) key verification key, a message, and a signature
- Provides:
 - Authentication, Data integrity, Non-Repudiation

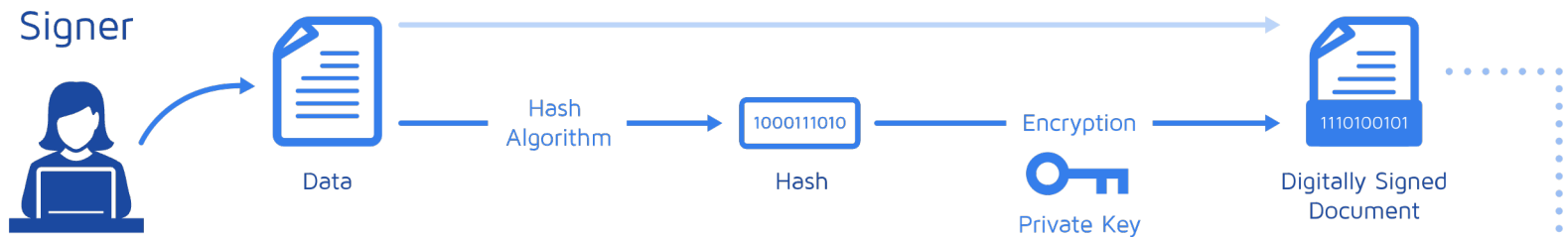


Key-pair Terminology

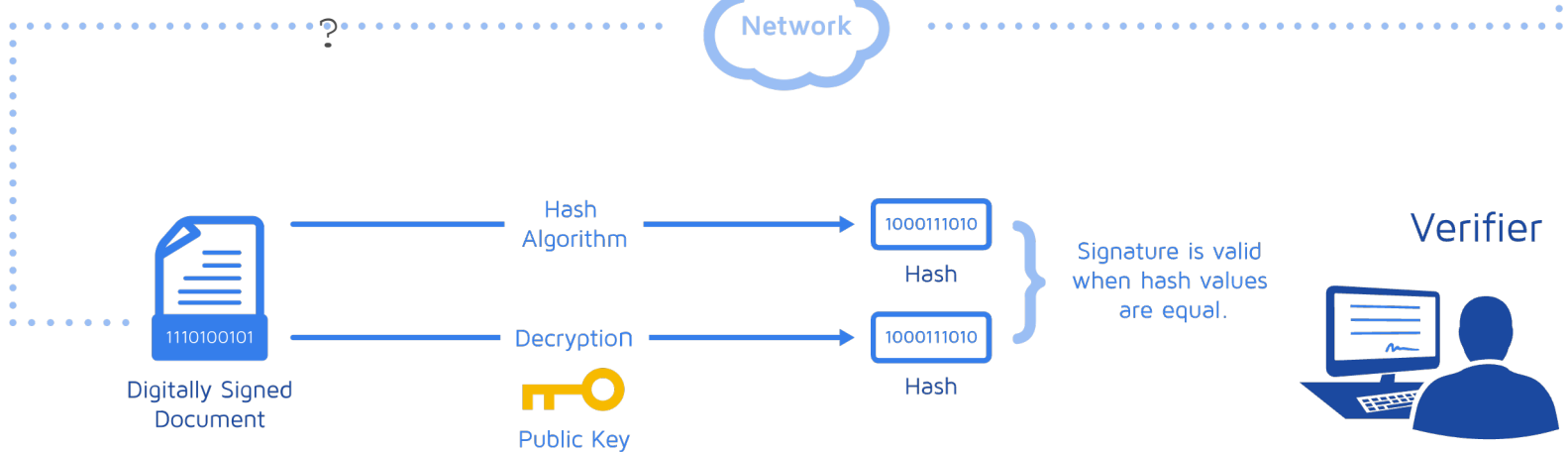
	Encryption	Digital Signatures
Public key	Encryption key	Verification key
Private key	Decryption key	Signing key



How does Digital Signatures work?



How does Digital Signatures work





Digital Signature Scheme

- Digital Signature based on RSA:

A triple $(\text{Gen}, \text{Sign}, \text{Ver})$

- $\text{Gen}(\text{len})$: generate a key-pair (pk, sk)
- $\text{Sign}(m; sk)$: sign a message m with private key sk ,
output signature s
- $\text{Ver}(m, s; pk)$: verify signature s with public key pk

Lecture 5:

Elliptic Curve Cryptography





Outline

➤ Elliptic Curves Cryptography

1. Basic Elliptic Curves
2. Addition and Multiplication on Elliptic Curve
3. Elliptic Curves Discrete Logarithm Problem (ECDLP)
4. Elliptic Curves Diffie Hellman (ECDH)
5. Elliptic Curve Digital Signature Algorithm (ECDSA)

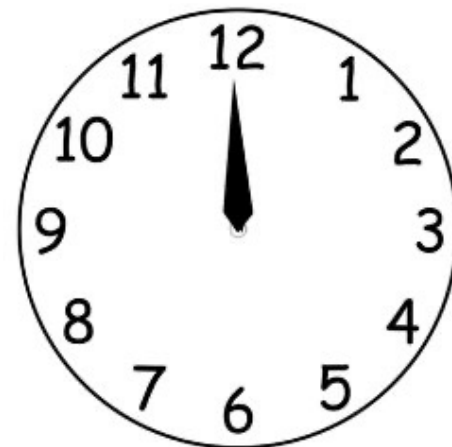
➤ Reading Session



Trapdoor/One-way Function

- In Diffie-Hellman Key Exchange Protocol:
 - Discrete Logarithm Problem $s = g^n \bmod p$
 - Without modular arithmetic operation $s = g^n$
- In RSA:
 - Factoring problem
 - Choose $n = pq$, the product of two primes.
 - Then $\phi = (p-1)(q-1)$ which can only be calculated if n is factored.
 - We have to know both p and q to calculate ϕ .
- In Elliptic Curve???

Modular/Clock Arithmetic



Modulus 12



Discrete Logarithms in Finite Fields

$$F = \{1, 2, 3, \dots, p-1\}$$

Pick a secret, random x from F



Alice

Compute:

$$k = (g^y)^x = g^{xy} \bmod p$$

$$A = g^x \bmod p$$

$$B = g^y \bmod p$$

Pick secret, random y from F



Bob

Compute:

$$k = (g^x)^y = g^{xy} \bmod p$$

Trudy knows p, g, A and B

She has to compute k from A and B without knowing x and y ...

She faces the **Discrete Logarithm Problem** in finite fields



Security of Diffie Hellman

- The Diffie-Hellman algorithm relies on the hardness of the discrete logarithm problem for its security.
- The discrete logarithm problem states that if we know p, g and public key $A = g^x \% p$, then it is computationally infeasible to calculate the private key a .
- It can be shown that this problem is equivalent to the factoring problem.
 - If a fast algorithm for factoring is discovered then there exists an equivalent fast algorithm for solving the discrete logarithm problem, and vice versa.

1. Basic Elliptic Curve





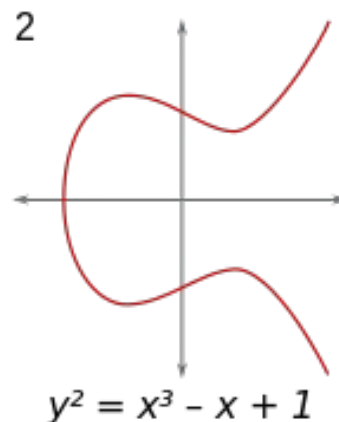
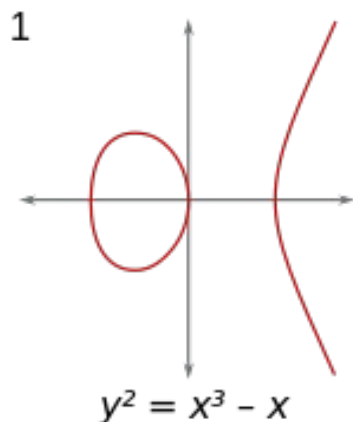
Elliptic Curves

- Basic Elliptic Curves:

$$y^2 = x^3 + Ax + B$$

$$(4A^3 + 27B^2 \neq 0) \text{ (Weierstrass Form)}$$

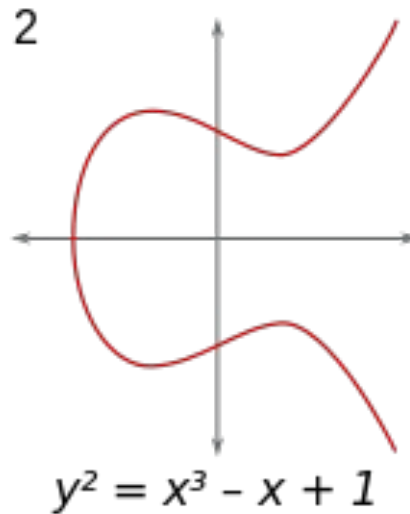
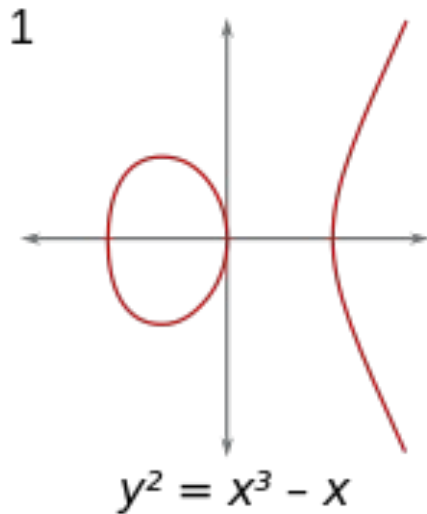
- The values A, B, x, y come from some sets, usually a field





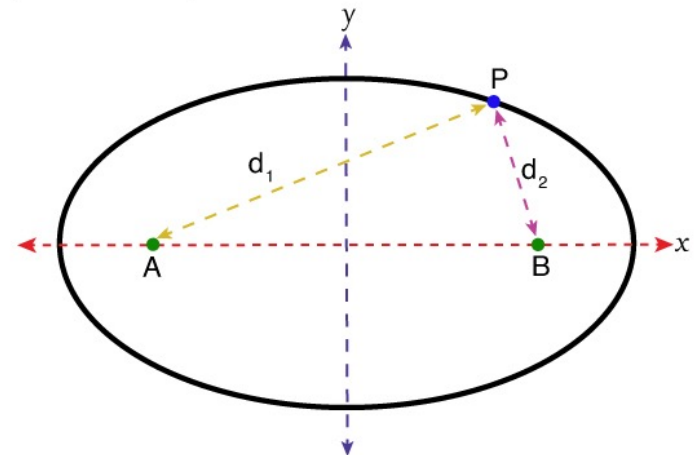
Elliptic Curves

- Elliptic Curves are different from Ellipse



Ellipse

MATH
MONKS



Here,

P is any point on ellipse,

A & B are the foci,

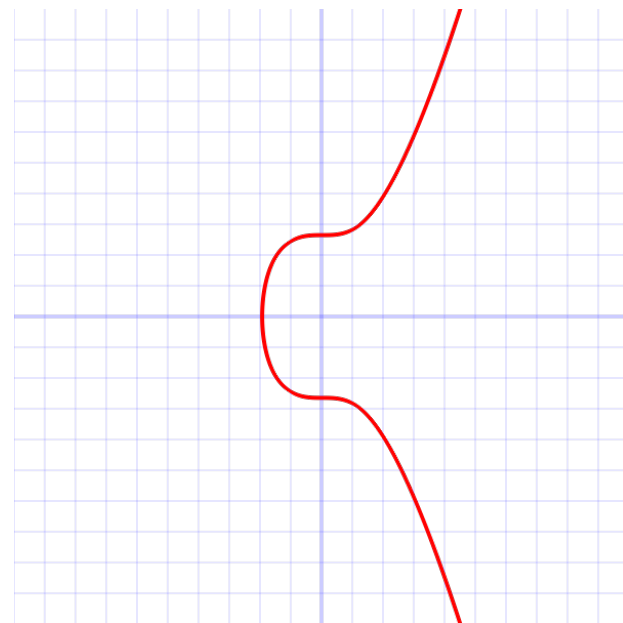
$PA = d_1$, $PB = d_2$, & $d_1 + d_2 = \text{constant}$



Elliptic Curve in Bitcoin

Elliptic Curve used in
Bitcoin

$$y^2 = x^3 + 7$$



<https://en.bitcoin.it/wiki/Secp256k1>

- This is a graph of *secp256k1*'s elliptic curve $y^2 = x^3 + 7$ over the real numbers.
- Note that because *secp256k1* is defined over the finite field $\mathbb{Z}_p$, its graph will in reality look like random scattered points, not anything like this.



Definition of EC

- An **elliptic curve** over a field L is a non-singular cubic curve in two variables, $f(x, y) = 0$ with a rational point (which may be a point at infinity).
- The field L is usually taken to be the complex numbers, reals, rationals, algebraic extensions of rationals, p-adic numbers, or a Finite Field.
- Elliptic curves groups for cryptography are examined with the underlying fields of F_p (where $p > 3$ is a prime) and F_{2^m} (a binary representation with 2^m elements).



EC on a Finite Set of Integers

➤ Consider $y^2 = x^3 + 2x + 3 \pmod{5}$

$$x = 0 \Rightarrow y^2 = 3 \Rightarrow \text{no solution} \pmod{5}$$

$$x = 1 \Rightarrow y^2 = 6 = 1 \Rightarrow y = 1, 4 \pmod{5}$$

$$x = 2 \Rightarrow y^2 = 15 = 0 \Rightarrow y = 0 \pmod{5}$$

$$x = 3 \Rightarrow y^2 = 36 = 1 \Rightarrow y = 1, 4 \pmod{5}$$

$$x = 4 \Rightarrow y^2 = 75 = 0 \Rightarrow y = 0 \pmod{5}$$

➤ Then points on the elliptic curve are: $(1, 1)$ $(1, 4)$
 $(2, 0)$ $(3, 1)$ $(3, 4)$ $(4, 0)$ and the point at infinity: ∞

➤ Using finite fields, we can form an Elliptic Curve Group for public-key cryptography where we have a trapdoor function as the DLP problem.



Points on Elliptic Curve

- Elliptic Curve over field L

$$E(L) = \{\infty\} \cup \{(x, y) \in L \times L \mid y^2 + \dots = x^3 + \dots\}$$

- Forms an Abelian group
- Symmetric about the x -axis
- Points of the curve: $P = (x, y)$ with $x, y \in L$ and a point ∞
 - Point at infinity ∞ acting as the Identity Element in Abelian group

2. Addition and Multiplication on Elliptic Curve





The Abelian Group

- Abelian group: a set of elements together with an operation (+) to form another element. Here, given two points P, Q in $E(F_p)$, there is a third point, denoted by $P+Q$ on $E(F_p)$
- The following relations hold for all P, Q, R in $E(F_p)$:
 1. $P + Q = Q + P$ (**commutativity**)
 2. $P + (Q + R) = (P + Q) + R$ (**associativity**)
 3. $P + O = O + P = P$ (**existence of an identity element**)
 4. There exists $(-P)$ s.t $-P + P = P + (-P) = O$ (**existence of inverses**)



The Abelian Group

- Consider elliptic curve

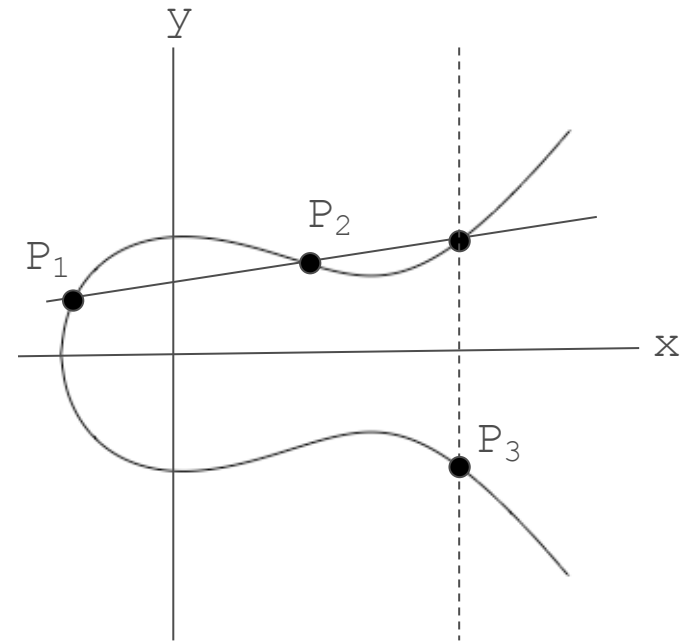
$$\text{EC: } y^2 = x^3 - x + 1$$

- If P_1 and P_2 are on EC, we can define

$$P_3 = P_1 + P_2$$

as shown in the picture.

- Addition operation (+) is all we need



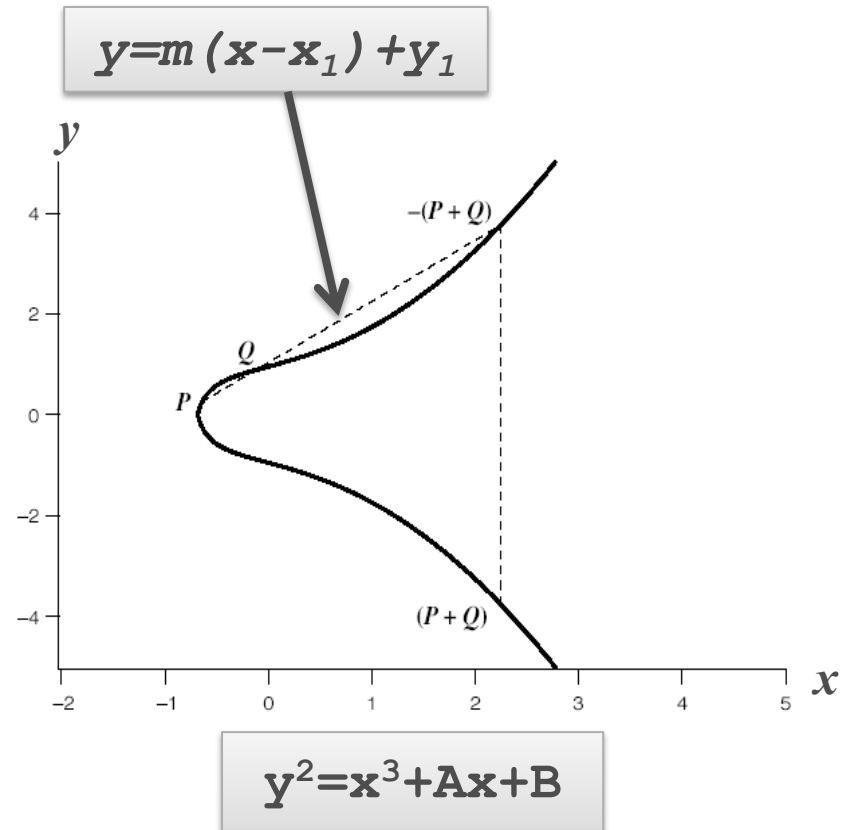


Addition of two Points on EC

Consider two points P and Q

$$P = (x_1, y_1), Q = (x_2, y_2)$$

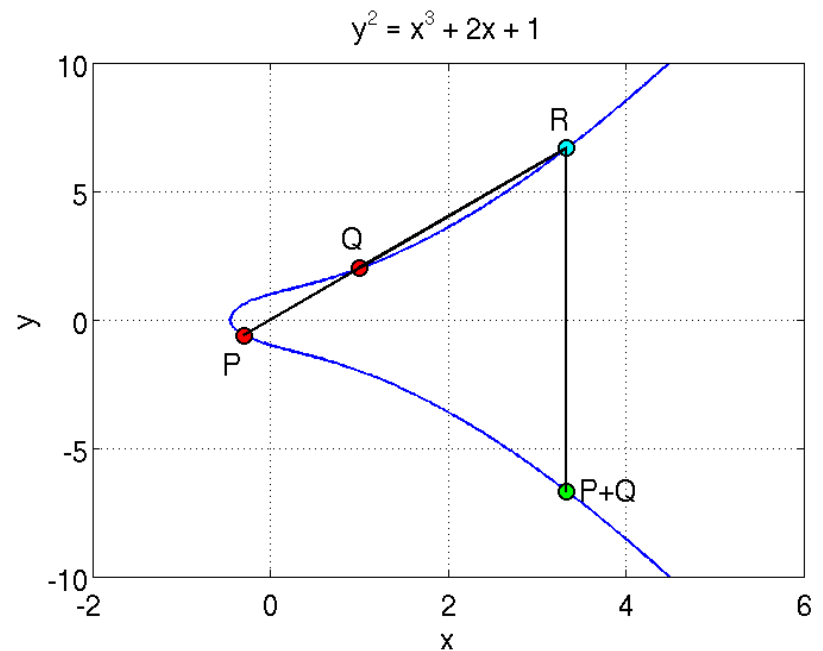
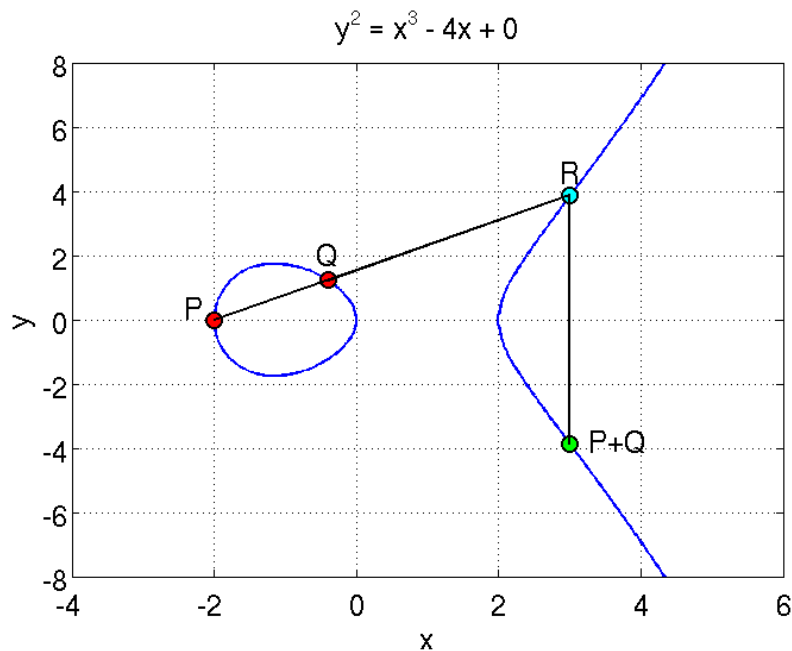
$$R = (P + Q) = (x_3, y_3)$$





Addition of two Points on EC

- P and Q are added to obtain $P+Q$, which is a reflection of R along the x -axis

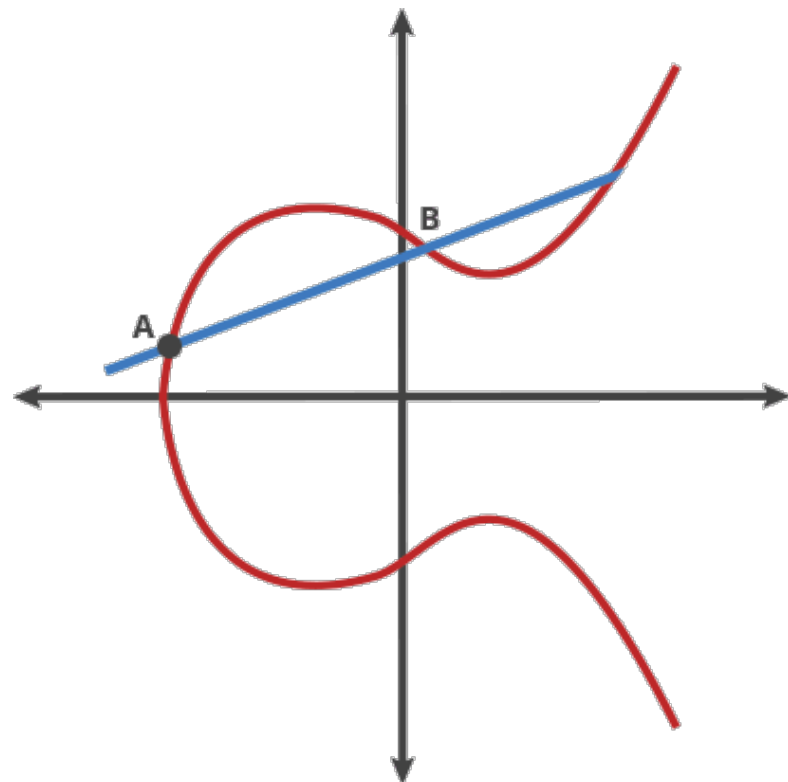




Addition of two Points on EC

➤ Using our definition of Addition:

1. $P + Q$ is well defined
2. $P + Q = Q + P$
3. $P + (Q + R) = (P + Q) + R$
4. $P + O = P$
5. $-P = P \# (O \# O)$



<https://blog.cloudflare.com/a-relatively-easy-to-understand-primer-on-elliptic-curve-cryptography>



Doubling a Point on EC

➤ In case $P = Q$, we have

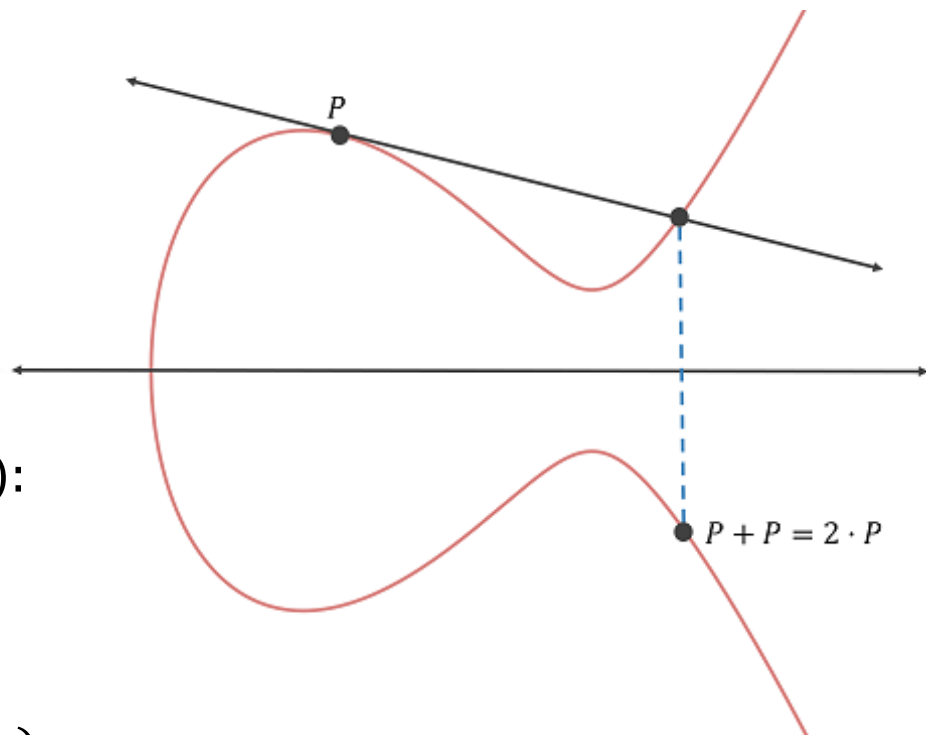
$$2y \frac{dy}{dx} = 3x^2 + A$$

$$\Rightarrow m = \frac{dy}{dx} = \frac{3x_1^2 + A}{2y_1}$$

If, $y_1 \neq 0$ (since then $P_1 + P_2 = \infty$):

$$0 = x^3 - m^2x^2 + \dots$$

$$\Rightarrow x_3 = m^2 - 2x_1, y_3 = m(x_1 - x_3) - y_1$$





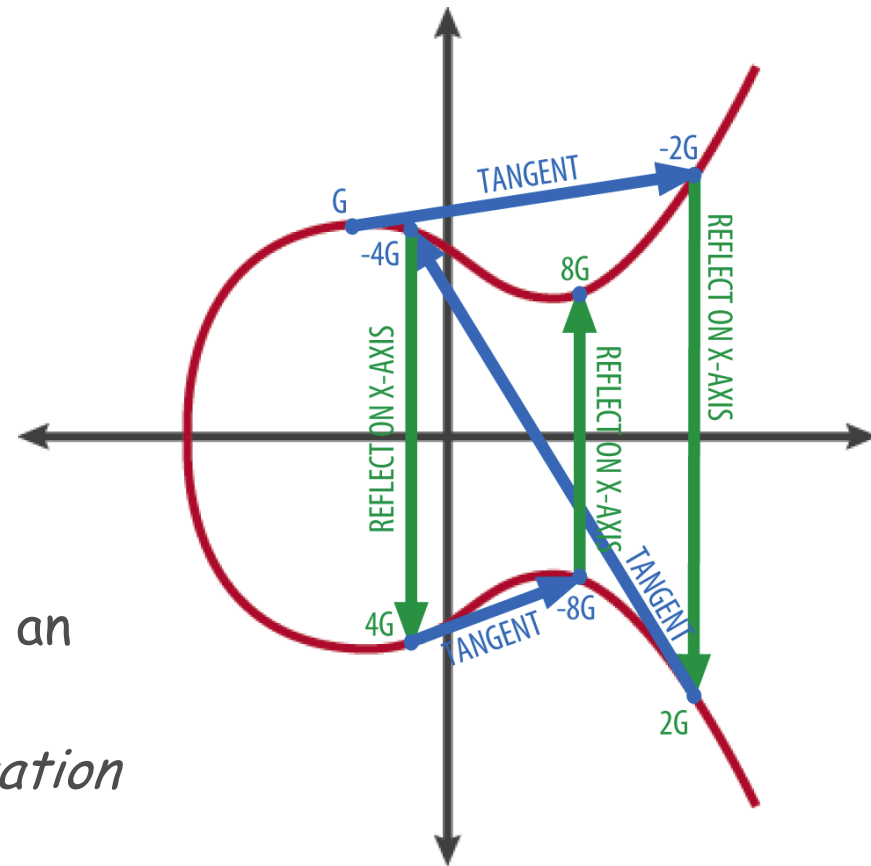
Multiplication of a Point

➤ We define mP :

- $2P = P + P$
- $3P = P + P + P$
- $mP = P + P + \dots + P$

m : order of P

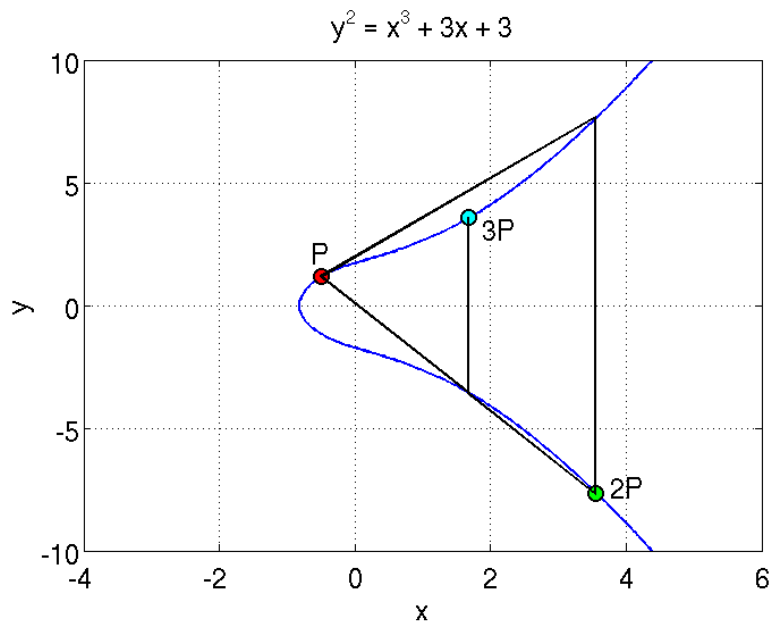
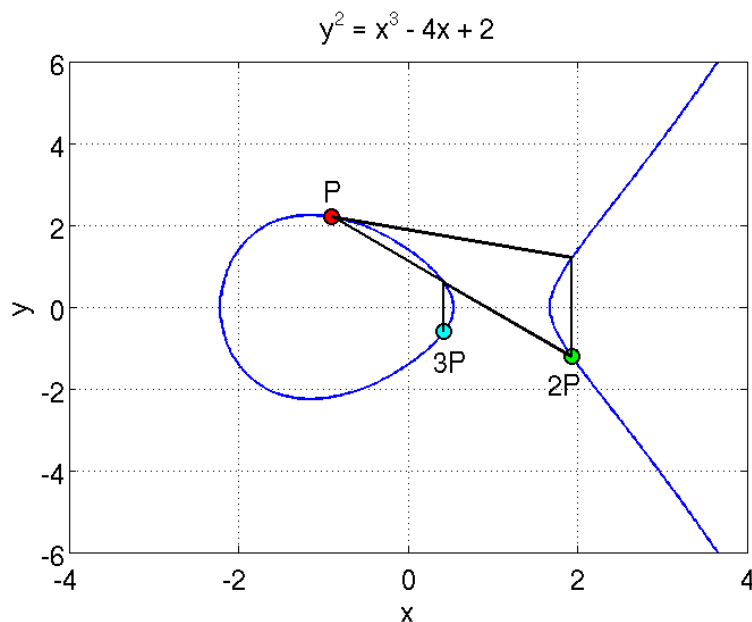
- No matter how big m is, there is an efficient way to calculate mP .
- It is also called *Scalar Multiplication* on EC





Multiplication of a Point

- A tangent at P is extended to cut the curve at a point; its reflection is $2P$. Adding P and $2P$ gives $3P$
- Similarly, such operations can be performed as many times as desired to obtain $Q = mP$



3. Elliptic Curve Discrete Log Problem (ECDLP)





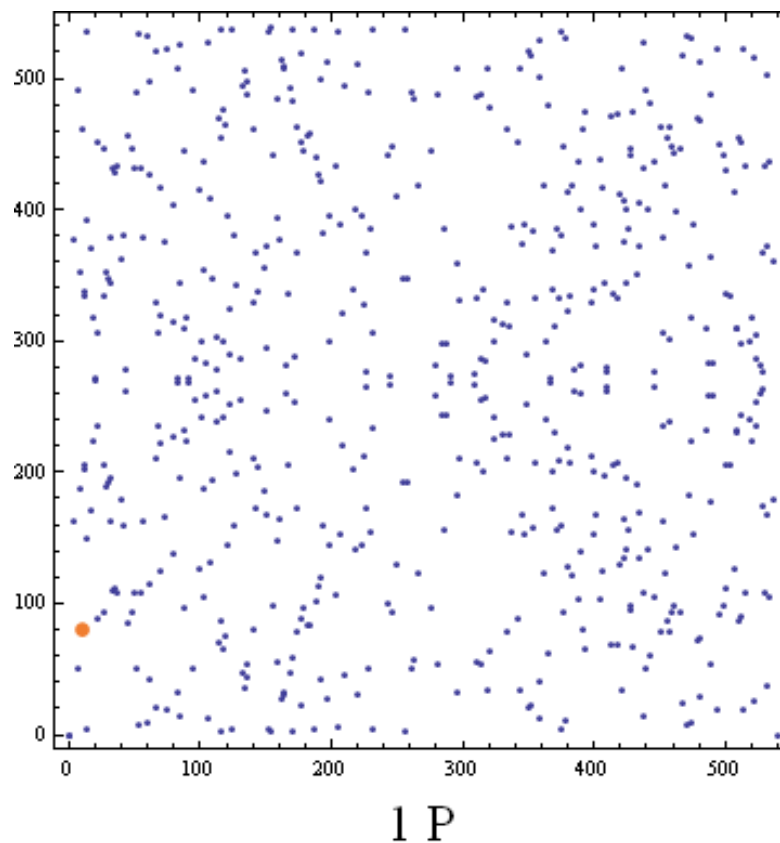
EC Discrete Log Problem

- Combined with "Integer mod p "
- **ECDLP**: Begin with an $EC \bmod p$, let P be a point and let Q be a multiple of P : $Q = mP$. The ECDLP is to find the value of m given Q and P .
- We can simply calculate $2P$, $3P$, $4P$, etc. But if p and m are large numbers, this could take trillions of years (Exponential running time)
- Until now, we do not know of a fast way to solve **ECDLP**.



Example: EC mod 541

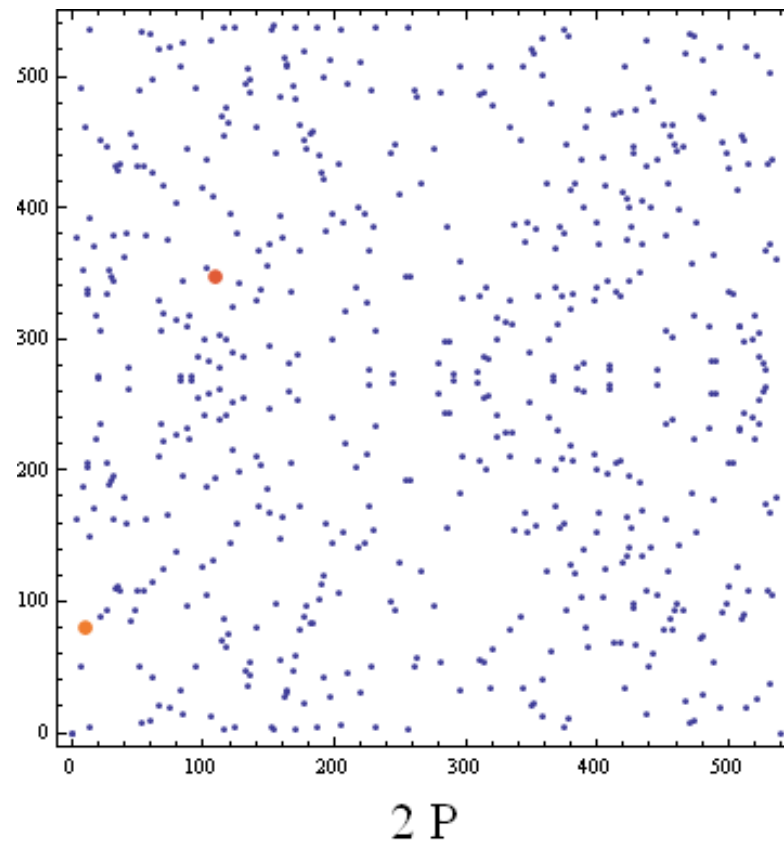
EC: $y^2 = x^3 - x$ with $p = 541$, point $P = (10, 80)$





Example: EC mod 541

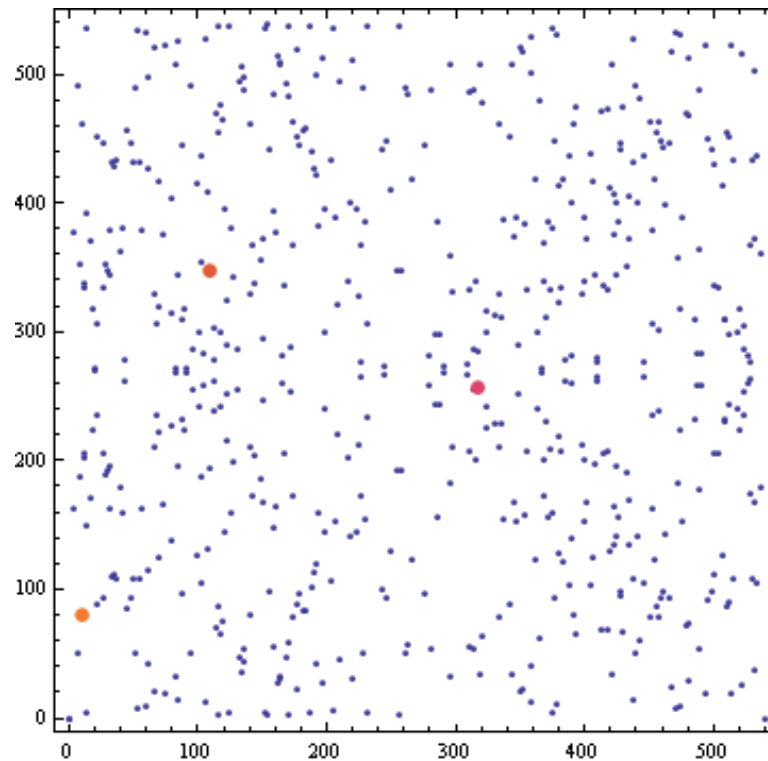
EC: $y^2 = x^3 - x$ with $p = 541$, point $P = (10, 80)$





Example: EC mod 541

EC: $y^2 = x^3 - x$ with $p = 541$, point $P = (10, 80)$

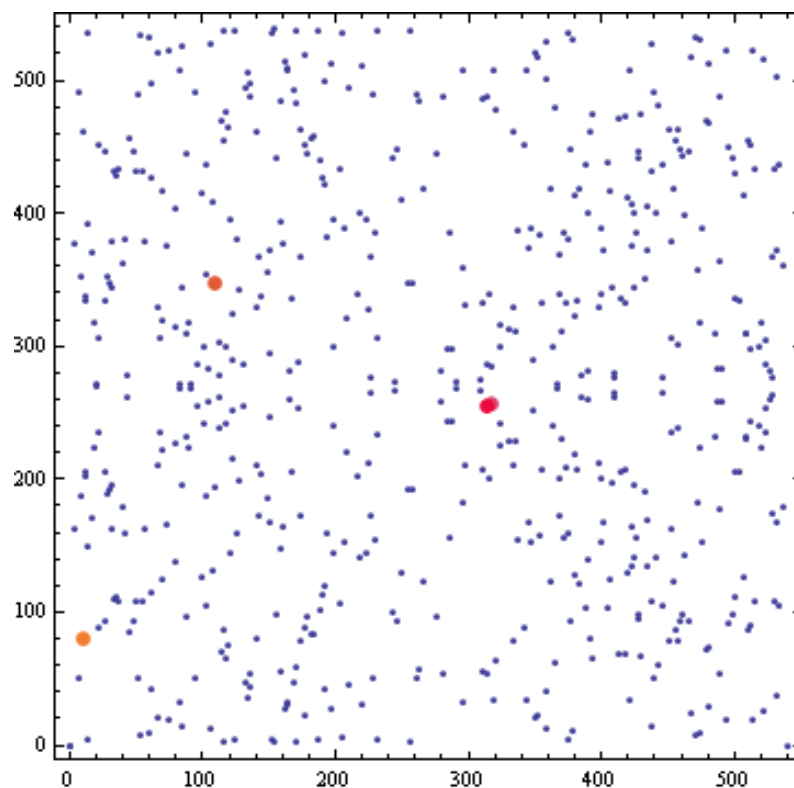


3 P



Example: EC mod 541

EC: $y^2 = x^3 - x$ with $p = 541$, point $P = (10, 80)$

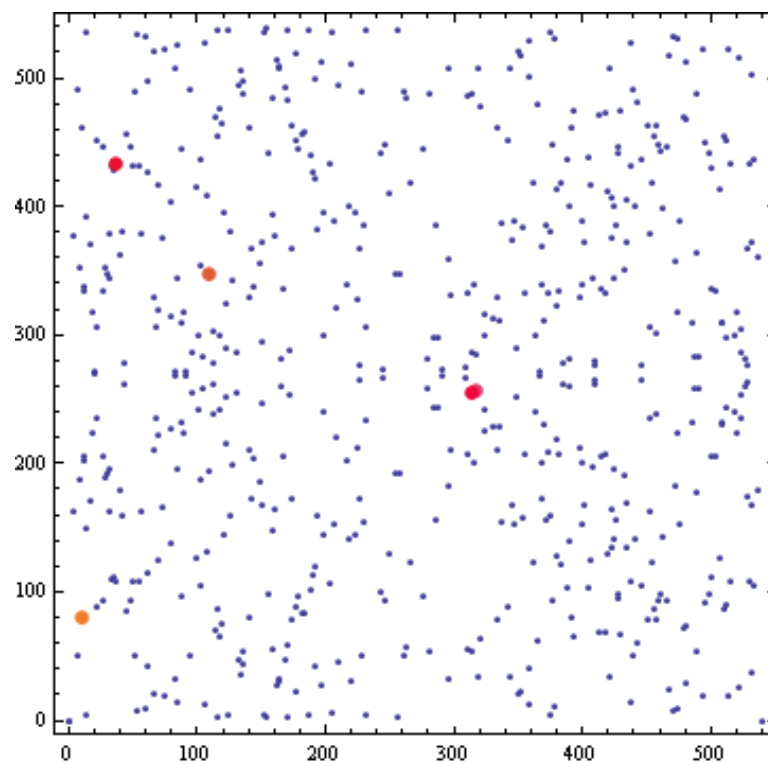


$4P$



Example: EC mod 541

EC: $y^2 = x^3 - x$ with $p = 541$, point $P = (10, 80)$

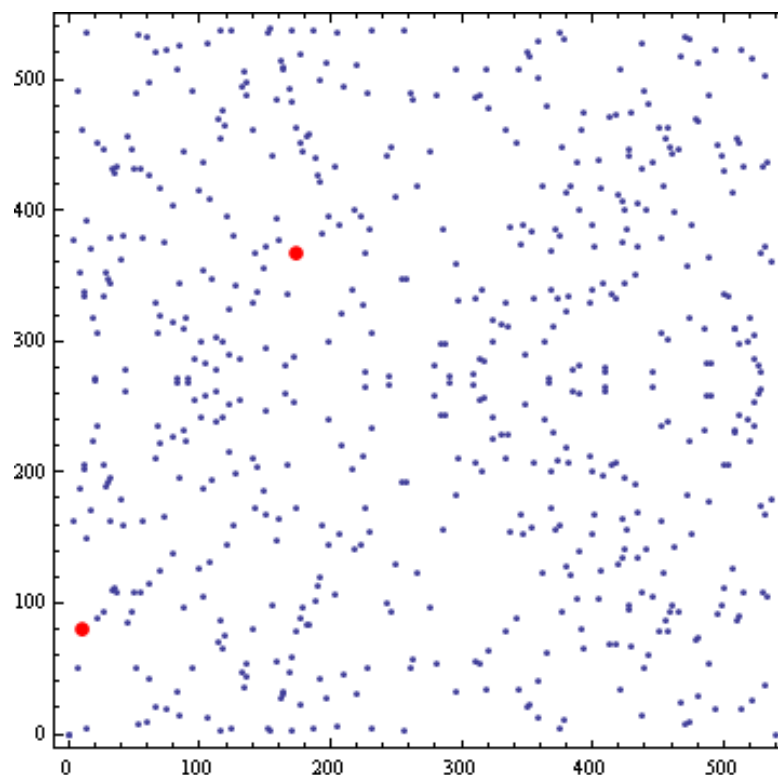


5 P



Elliptic Curve Discrete Log Problem (ECDLP)

EC: $y^2 = x^3 - x$ with $p = 541$, point $P = (10, 80)$



$$(173, 368) = m(10, 80)$$



Elliptic Curve Diffie-Hellman

4. Elliptic Curve Diffie-Hellman (ECDH)





EC Discrete Log Problem

- There is an efficient way to calculate the multiplication on EC:
 $Q = mP$, given m and P
- The ECDLP is to find the value of m , given Q and P s.t $Q = mP$
 - Basically, we do not know of a fast way to solve ECDLP.
- This will be good for the trapdoor function (with modular p)
 - m will be the secret information, Q and P and the modulus prime p can be publicly available



ECDH Key Exchange Protocol

- Alice and Bob desire to establish a shared secret key k .
- Alice and Bob agree on an EC , a large prime p (called the multiplicative order), and a point B on the curve (called the Generator or the Base-point)
 - Called Domain Parameters
- Trudy knows the curve, the point, and the prime number p



ECDH Key Exchange Protocol

- Alice secretly picks a large number p_A . Bob secretly picks a large number p_B . Alice computes $Q_A = p_A B$; Bob computes $Q_B = p_B B$. They exchange the points Q_A and Q_B .
 - p : private key
 - $Q = pB$: public key
- Alice computes $p_A Q_B = p_A p_B B$. Bob computes $p_B Q_A = p_B p_A B$. Both use the x-value of $p_A p_B B$ for the key k .
- The shared secret key k is then used for symmetric encryption such as DES and AES



ECDH Example

1. Let's use $y^2 = x^3 - x$ with $p = 541$, $B = (10, 80)$.
2. Alice picks $p_A = 20$. Bob picks $p_B = 103$.
3. $Q_A = 20 \times (10, 80) = (519, 241)$.
4. $Q_B = 103 \times (10, 80) = (85, 345)$.
5. When Alice gets Q_B , she finds $20 \times Q_B = (353, 158)$.
6. When Bob gets Q_A , he finds $103 \times Q_A = (353, 158)$.
7. They both use $k = 353$ for their secret key.



Diffie-Hellman: Finite Fields and ECs

Alice

Choose
 $a \in_R \{0, \dots, q-1\}$

$$\xrightarrow{A = g^a \pmod{p}}$$

Compute:
 $K = B^a \pmod{p}$

Same K :

$$A^b = (g^a)^b = g^{ab} = (g^b)^a = B^a$$

Bob

Choose
 $b \in_R \{0, \dots, q-1\}$

$$\xleftarrow{B = g^b \pmod{p}}$$

Compute:
 $K = A^b \pmod{p}$

Alice

Choose
 $a \in_R \{0, \dots, q-1\}$

$$\xrightarrow{A = aP}$$

Compute:
 $K = bA$

Same K :

$$bA = baP = abP = aB$$

Bob

Choose
 $b \in_R \{0, \dots, q-1\}$

$$\xleftarrow{B = bP}$$

Compute:
 $K = aB$



Why ECC

1. ECC: Shorter key-length

Symmetric Key Length	Standard Asymmetric Key Length	Elliptic Curve Key Length
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	512



Why ECC

1. ECC: Shorter key-length
2. Fast Computation (keyGen and Digital Signature)

What are the disadvantages of ECC?

1. ECDLP is not mature yet! Might have an unknown weakness
2. Heavily rely on RNG
 - If RNG is compromised, then the key can be figured out

5. Elliptic Curve Reading Session





Reading List

1. https://en.wikipedia.org/wiki/Elliptic_Curve_Digital_Signature_Algorithm
2. https://en.bitcoin.it/wiki/Elliptic_Curve_Digital_Signature_Algorithm
3. <https://www.cs.miami.edu/~burt/learning/Csc609.142/ecdsa-cert.pdf>
 - This file is uploaded on Moodle as well



Summary

- Elliptic Curves Cryptography
 - Basic Elliptic Curves
 - Addition and Multiplication on Elliptic Curve
 - Elliptic Curves Discrete Logarithm Problem (ECDLP)
 - Elliptic Curves Diffie Hellman (ECDH)



Experiment with OpenSSL command line

- OpenSSL contains a large set of pre-defined curves that can be used. The full list of built-in curves can be obtained through the following command:

```
$ openssl ecparam -list_curves
```

- An EC Parameters file contains all of the information necessary to define an Elliptic Curve that can then be used for cryptographic operations (for OpenSSL this means ECDH and ECDSA):

```
$ openssl ecparam -name secp256k1 -out  
secp256k1.pem
```




Experiment with OpenSSL command line

- Keys can be generated from the `ecparam` command, either through a pre-existing parameters file or directly by selecting the name of the curve. To generate a key pair from a pre-existing parameters file use the following:

```
$ openssl ecparam -in secp256k1.pem -genkey -noout  
-out secp256k1-key.pem
```

- Or to do the equivalent operation without a parameters file use the following:

```
$ openssl ecparam -name secp256k1 -genkey -noout -  
out secp256k1-key.pem
```



Experiment with OpenSSL command line

- Remember, the command generates the keypair, both private and public keys.

```
$ openssl ecparam -in secp256k1.pem -genkey -noout -out secp256k1-key.pem"
```

- The public key can be obtained using this command:

```
$openssl ec -in ecprivkey.pem -pubout -out ecpubkey.pem
```

- Just take a look at the key file, which contains both information on private and public keys (and the curve as well):

```
$openssl ec -in secp256k1-key.pem -noout -text
```



Experiment with OpenSSL command line

- Information on the parameters that have been used to generate the key are embedded in the key file itself.

```
$ openssl ecparam -in secp256k1.pem -text -noout
```

- This will simply confirm the name of the curve in the parameters file. If you wish to examine the specific details of the parameters associated with a particular named curve then this can be achieved as follows:

```
$ openssl ecparam -in secp256k1.pem -text -param_enc explicit -noout
```